AI-Assisted Coding with Claude Code

Specifying, delegating, and verifying work with an AI agent

https://github.com/andrewstellman/ai-training

Andrew Stellman

- Over 20 years training people on software development
- Author of 6 O'Reilly books including *Head First C#*
- Full-time software developer and team lead
- Passionate about all things code.

What's in this course

A really quick overview of what we're going to learn…
and then we'll jump right into it.

“Just enough to be dangerous…”

- **Specifying:** Learn to describe a piece of work clearly enough that an agent can carry it out.
- **Delegating:** Hand real work to Claude Code and let it read, write, and run your code.
- **Verifying:** Check what came back, and in Part 2 put a second agent to work reviewing it.
- **Practical experience:** Hands-on exercises analyzing code, delegating a change, and building a small project.

This is not a course in advanced AI!

This Course Is For:

- Developers who want to start working with an AI agent, and haven't yet.
- Anyone who's used AI chatbots for code and wants to know what changes when the AI can act on your project directly.
- Developers who want habits they can put to work this afternoon.

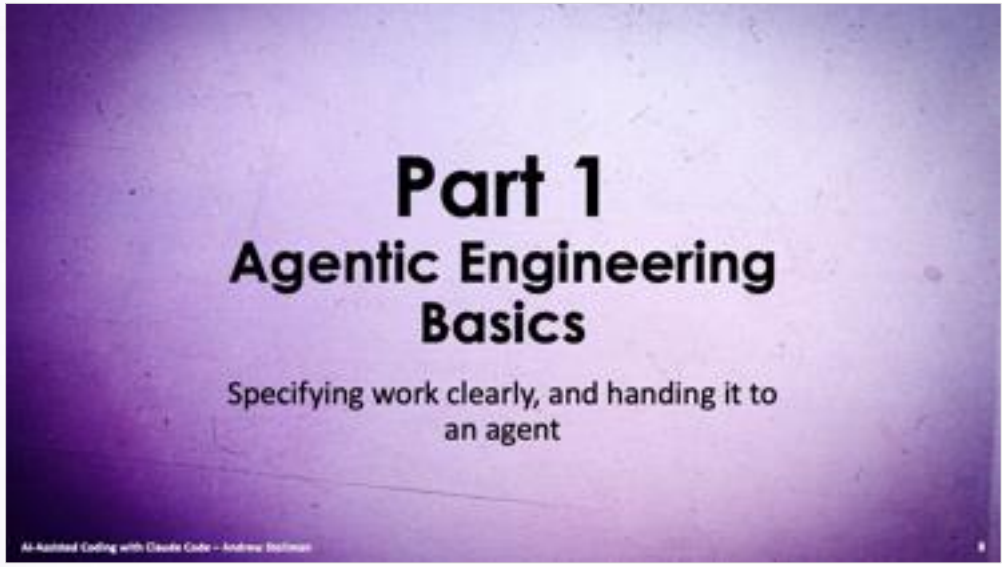
This Course Is Not For:

- Experts in AI or machine learning seeking advanced techniques.
- Those looking for in-depth theory on neural networks or deep learning.
- Developers looking for advanced automation, like using subagents, writing MCP servers, or running agents in CI/CD.

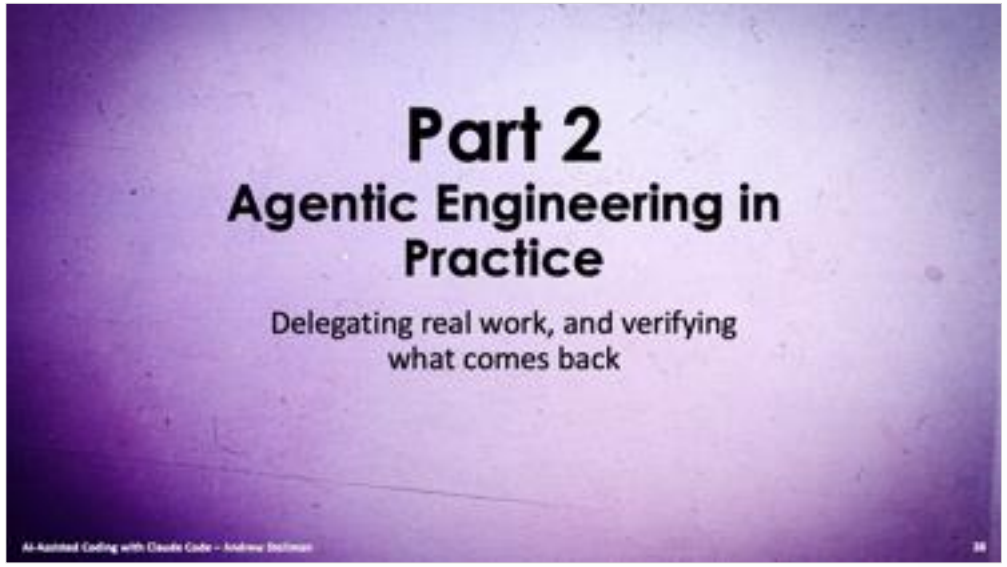
How this course is structured

- In the main **Presentation** sections, I'll do teaching and give demos using Java, Python, or C#. You're encouraged to follow along on your own.
- In the **Exercise** sections, I'll give you time to keep working on your own. You can download the code and slides from the GitHub page for this course: https://bit.ly/coding-ai-course
- This course is divided into two parts with a short break between them. Each part ends with a **Q&A** section where I answer your questions.

How this course is structured



We'll start with the basics of agentic engineering: what an agent is, and how to specify work clearly enough to hand over. Then we'll get hands-on analyzing and improving code.



We'll put it into practice. You'll delegate real work to Claude Code, verify what it did (including spinning up a second agent to review it), and build a small project of your own.

Part 1
Agentic Engineering Basics

Specifying work clearly, and handing it to an agent

Introducing Generative AI

Understanding the AI inside your agent – its potential and its limitations

What is generative AI?

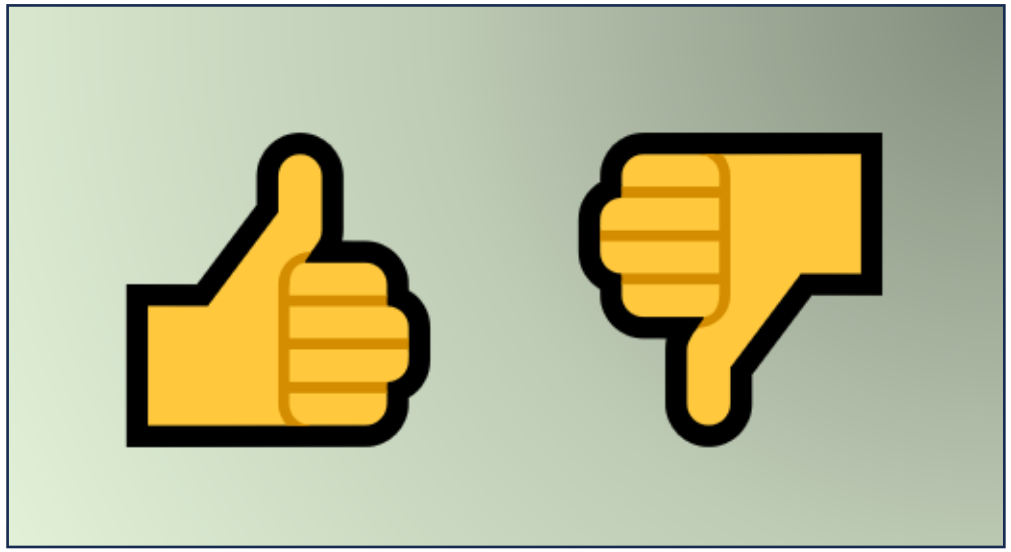
Generative AI refers to AI models that can generate text, images, or code, by learning patterns from ***existing data***.

- You can use generative AI to create stories, articles, song lyrics, poems, and even code based on a prompt.
- In coding, generative AI can help write and refactor code, generate documentation, and even suggest improvements.

*The models behind tools like Claude Code are trained on **billions of lines of Java, Python, and C# code**, so they can tap a vast knowledge base to generate code in response to your prompts.*

Pulse Check

Who here has used a Generative AI chatbot like Claude, ChatGPT, Gemini, or Microsoft 365 Copilot?



What is an AI agent?

An AI agent does things. It reads your files, changes them, runs commands, and reads the output.

- What makes it an agent is the loop: it acts, reads the result, decides what to do next, and repeats until the job is done.
- It decides a lot along the way: which files to open, which approach to take, when it's done.

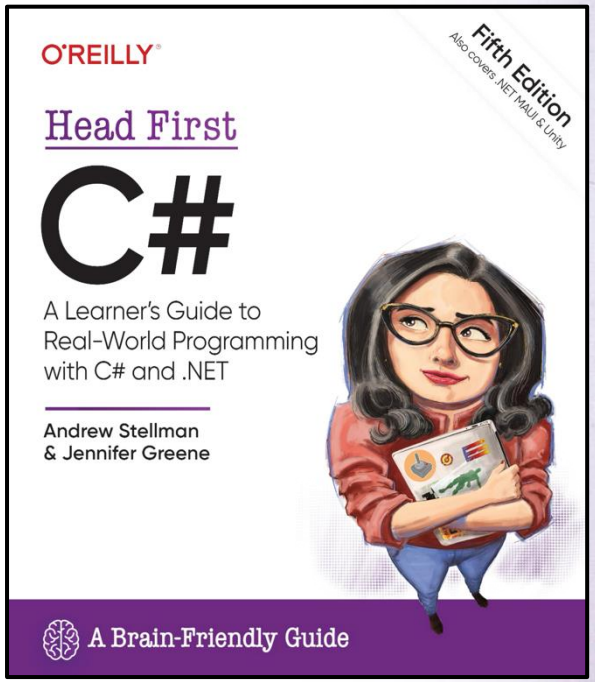
Claude Code is an agent that runs in your terminal. You say what you want, it works out how, and you'll see the loop on screen in every demo today.

What's different about working this way

- **You work in a terminal.** Claude Code runs there, and your editor stays open beside it, so you'll still read and review your code the way you always have.
- **You describe the work; Claude Code does the work.** Your time goes into deciding what you want, and deciding whether you got it.
- **You won't watch every line get typed.** If that feels strange, you're not alone. A big part of this course is getting comfortable with it *so you can start using it effectively today*.
- **You're always in control.** You choose whether it asks before every change or runs on its own with a safety check, and Esc stops it at any time.

AI seems like magic (but it's not)

- When I was writing the latest edition of Head First C# back in 2023, I used multiple AI chatbots to test the exercises.
- I pasted the instructions and code for the exercise into the chat as its prompt (the instruction for an AI to generate specific output), and then copied the code back into my IDE to test it.
- If the AI chatbot was able to generate a correct answer that worked, it meant that I gave my readers all the information that they needed to complete the exercise.



Demo: From C# to Python with Claude Code

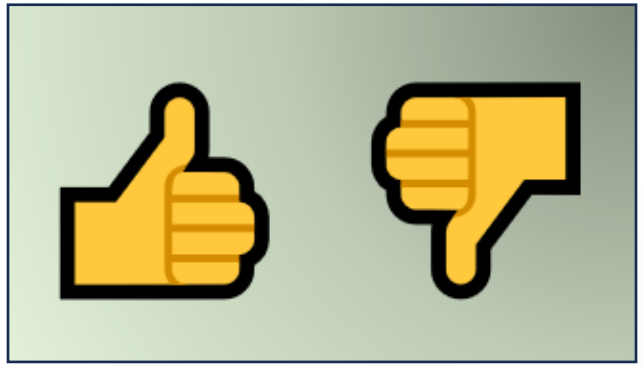
- I'm going to take an exercise from Head First C# and use Claude Code to build a working Python app from it.
- First, I'll paste the C# class into Claude Code and ask it to explain how it works.
- Then I'll ask Claude Code to convert it to Python and build the full application, even though the original instructions were written for C# developers.
- Watch the loop while it works: every file it reads, every change it makes, and every command it runs prints on screen.
- The exercise instructions aren't perfectly formatted: the indents are off, there are annotations and callout boxes mixed in. And that's okay! AI is really good at working around messy inputs.
- Once the app is running, I'll push Claude Code further and ask it to add some art.

What happens if we ask a simple question about code?

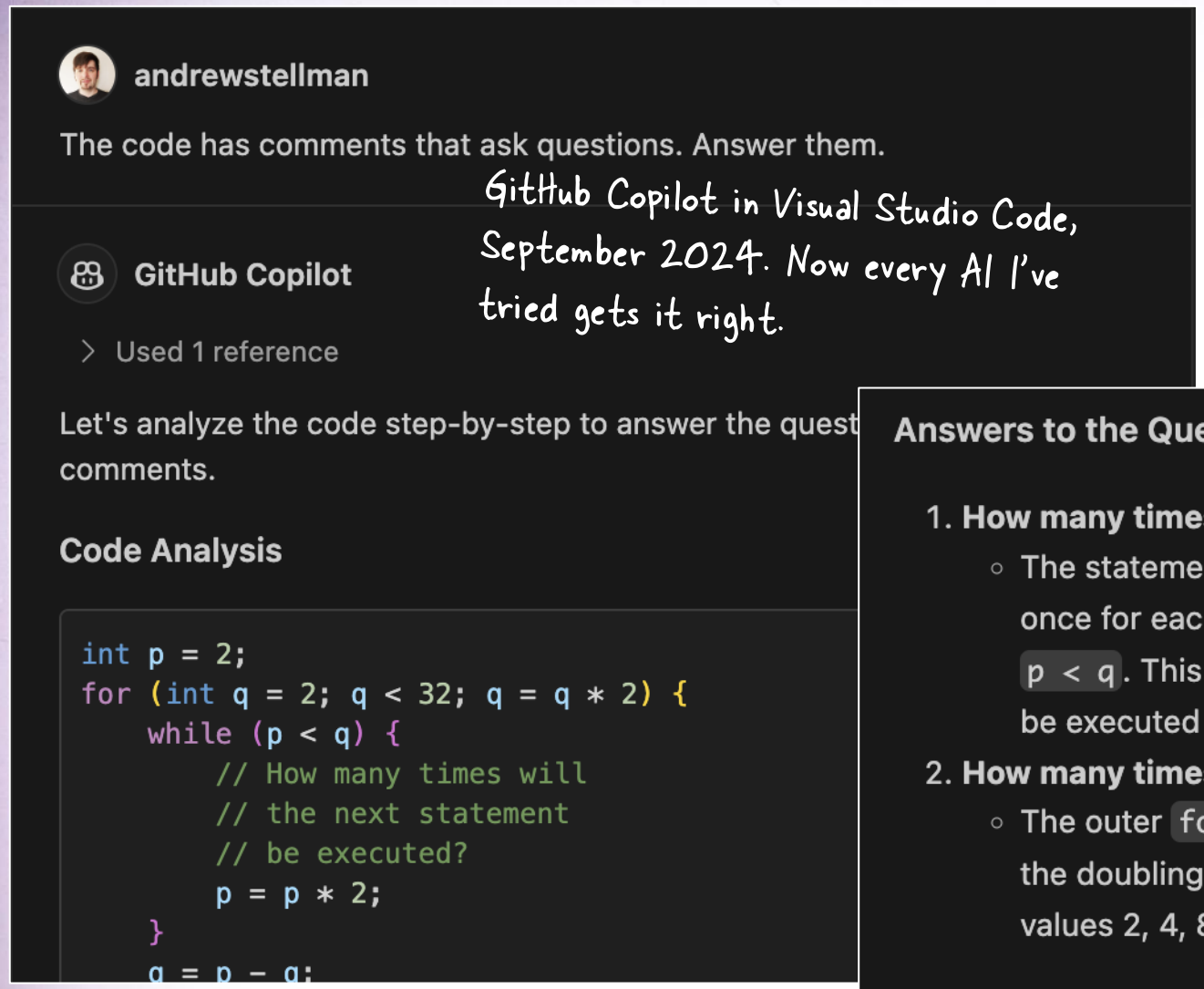
- This is an exercise from Chapter 2 of *Head First C#*. People who just learned about loops usually get it right. How do you think an AI does?

```
int p = 2;
for (int q = 2; q < 32; q = q * 2) {
    while (p < q) {
        // How many times will
        // the next statement
        // be executed?
        p = p * 2;
    }
    q = p - q;
}
// How many times will the loop run?
```

Answer 👍 if you think it gets the answer **right**



Answer 👎 if you think it gets the answer **wrong**



I've asked many different AIs this question, and it wasn't until mid-2025 that I saw an AI give the right answer, and Claude got there first, by writing a script instead of reasoning about it. Try it yourself with **Claude Code** (and with any other AI you use).

Answers to the Questions

1. **How many times will the next statement (`p = p * 2`) be executed?**
   - The statement `p = p * 2` inside the `while` loop will be executed once for each iteration of the `for` loop where `q` is a power of 2 and `p < q`. This happens when `q` is 4, 8, 16, and 32. Therefore, it will be executed 4 times.
2. **How many times will the loop run?**
   - The outer `for` loop will run until `q` is no longer less than 32. Given the doubling of `q` each iteration, the loop runs 5 times (for `q` values 2, 4, 8, 16, and 32).

Thus, the answers are:

1. The statement `p = p * 2` will be executed 4 times.
2. The outer `for` loop will run 5 times.

Not the right answer!

GenAI isn't perfect (and that's okay!)

Start by understanding the capabilities and limitations of the AI inside your agent.

- **Mimicking Patterns:** AI models create responses by mimicking patterns from the vast amounts of text they've been trained on, aiming to sound like a natural conversation.
- **Code Generation:** They use patterns from many code examples to generate code that looks like what they have seen before.
- **Perceived Understanding:** It's easy to think that an AI understands the content it generates or is pulling correct answers from a database. In reality, it's crafting responses based on patterns it learned during training.

AIs don't inherently fact-check; they can produce convincing but sometimes incorrect or made-up answers. They do this a lot. ***A lot. ALL THE TIME.***

An example that shows how AI works

This screenshot is from 2024. ChatGPT now gives a more coherent answer.

A farmer needs to cross a river with a boat. He has a goat with him. The boat can only hold the farmer and one other item at a time. How does he get the goat to the other side of the river?

To get the goat across the river, the farmer can follow these steps:

1. **Take the goat across the river** and leave it on the other side.
2. **Return alone** to the original side.
3. **Take the boat back across the river** to the original side.
4. **Take the goat across the river** again.
5. **Leave the goat on the other side** and **return alone** to the original side.
6. **Finally, take the boat across the river** to the other side with the goat.

By repeating these steps, the farmer will successfully transport the goat across the river.

The current generation of AIs often generate a complicated answer to a question about getting a goat across a river, usually involving multiple unnecessary crossings.

The AI generates this weird answer because its training data included thousands of examples of the classic “River crossing puzzle” that involves getting a goat, cabbage, and a wolf across the river. It uses the solution to that puzzle as a pattern to generate its answer.

How to use AI safely: Trust but verify

AI doesn’t give you the right answer. It generates an answer that sounds right.

- **Verify Information Accuracy:** Always double-check facts and information provided by AI, as it can sometimes be incorrect or incomplete.
- **Test Generated Code:** Before using AI-generated code, thoroughly review and test it to ensure it works as expected and meets your requirements.
- **Understand Limitations:** Recognize that AI can misunderstand prompts or lack context, leading to unexpected results.
- **Critical Examination:** Treat AI responses as a starting point for further research and validation, not as the final word.

Go to https://bit.ly/coding-ai-course
for the code in this exercise. It also has
a full PDF of this slide deck.

Exercise

This is your chance to try asking Claude Code to work with some code. You'll need Claude Code installed and signed in. See the setup slide near the end of the deck.

Exercise: Use Claude Code to analyze Python, Java, or C# code (Part 1)

- Open a terminal in an empty folder and start Claude Code by running claude. Press shift-tab until the status bar says manual mode on.
- Go to https://bit.ly/coding-ai-course and copy either the Python, Java, or C# version of the Greeter class. Paste it into the prompt.
- Type \ then Enter twice to start a new line without sending the prompt
- Finish the prompt by asking the AI to explain what this code does. Use a prompt like this (replace [Python / Java / C#] with either Python , Java, or C#):

```
Can you explain what this [Python / Java / C#]
code does and how it works?
```

Ask it to suggest improvements (Part 2)

- Ask the AI to suggest a simple improvement to the code. You could use a prompt like this:

```
Can you suggest a simple improvement to make
this code better?
```

Take a minute and read the response.

How accurate was the AI's explanation?

Did the AI catch the null check and empty string handling?

Was the improvement suggestion relevant and useful?

AIs don't understand your code, but they sure look like they do

The AI is working from patterns it learned, but it does a really good impression of understanding your code… and that's good enough for it to be *really* useful.

An AI agent can explain your code

AI agents don't understand code the way you do, but they can still give you useful insights.

- **Fluent isn't the same as correct.** An explanation can sound confident and still be wrong, so treat it as a starting point rather than a verdict.
- **What they actually do:** They read the structure and logic of your code, then explain it, describe what functions are for, and flag potential issues.
- **Why that's useful:** An explanation you can check is faster than reading cold, both for you and for whoever works on the code next.

Demo: AI-assisted code review

- I'll start with code from Head First C# and use Claude Code to analyze it.
- First I'll ask the agent to run the app for me (it runs the command itself, right in the session). Then I'll ask it to explain how the code works, and what it used in the code to figure that out.
- Watch **which files it decides to read**. Nobody tells it where to look, and it's going to figure out more about this code than you might expect.
- Then I'll ask for a **code review**. You don't need one of those long viral code-review prompts, because a simple ask works.
- Finally, I'll ask the agent to **refactor and improve the code** based on its own suggestions.

How AI Agents "Read" Code

Here's what you just watched the agent do in the demo.

- **It went and looked.** Nobody told it which files mattered, so it searched the project, read the files it needed, and ran the app to see what it does.
- **It reasoned from clues.** It used the names, the structure, the comments, and the output it got back to explain code it had never seen before.

Bring your own critical thinking. *Judge the quality of what Claude Code tells you rather than simply accepting it. Does the explanation make sense? Does it check out against what you already know about the code?*

The basics of specifying what you want

Describing the work clearly to get the best results.

- Specifying is the art and science of describing a piece of work clearly enough that an agent can carry it out.
- A well-crafted prompt can significantly improve the quality and relevance of what you get back.
- A precise description gives it less to guess about, and fewer decisions to make on your behalf.

Essential tips for specifying clearly

Clear and detailed prompts get you better results from your AI.

- **Be Specific and Clear:** Clearly state what you're looking for. Instead of “clean this up,” say “Extract the duplicated validation in Program.cs into one method, and don’t change the behavior.”
- **Provide Examples:** Give the AI a scenario or example to show the AI what you’re working with. For example, “Given this code snippet [insert code], how can I optimize it for performance?”
- **Iterate, iterate, iterate:** Start with broader questions and then refine based on the AI's initial response. Keep refining your prompt until the AI gives you what you need.

Specifying is what makes refactoring work

Refactoring matters: It's so important for maintaining code quality and making sure your code is efficient, readable, and easy to change. AI is good at giving you suggestions that aren't immediately obvious. It can:

- **Identify Redundant Code:** Ask AI to review your code for redundancy: *"Do I have duplicated code in this file that can be moved to a method?"*
- **Improve Readability:** Request AI to make code more readable: *"Refactor this function to make it more understandable and maintainable."*
- **Improve Performance:** Use AI to suggest performance optimizations: *"How can I optimize this method to reduce its execution time?"*

Go to https://bit.ly/coding-ai-course
for the slides in this deck.

Exercise

Let's explore some of the more advanced analysis and refactoring capabilities of AI.

Exercise: Use Claude Code to analyze a more complex class (Part 1)

- Open a terminal in an empty folder and start Claude Code by running claude. Press shift-tab until the status bar says manual mode on.
- Go to https://bit.ly/coding-ai-course and copy the Python, Java, or C# code for the CustomSorter class. (Alternately, copy a class from your own code.) Paste it into the prompt.
- Type \ then Enter twice to start a new line without sending the prompt
- Finish the prompt by asking Claude Code to explain what this code does and how it works and press enter. Here's a prompt you can use:

```
Here's a [Python/Java/C#] implementation of a class.
Can you explain what this code does and how it works?
```

Ask the AI to add comments and suggest improvements (Part 2)

- Ask Claude Code to give potential improvements or optimizations for the code. Here's an example prompt that you can use:

  `What potential improvements or optimizations could be made to this CustomSorter implementation?`

- Ask Claude Code to add comprehensive comments to the class and its methods using docstrings (Python), JavaDoc (Java), or XMLDoc (C#) by giving it this prompt (replace *[docstrings/JavaDoc/XMLDoc]* to match your language):

  `Add [docstrings/JavaDoc/XMLDoc] comments to the code.`

- Ask Claude Code to generate a simple main block (Python), main method (Java) or top-level statements (C#) that demonstrates how to use the class:

  `Can you create [a __main__ block / a main method / top-level statements] to demonstrate the usage of this code?`

Refine and iterate (Part 3)

Let’s get some practice using Claude Code to analyze and explain complex code, so you can get some experience with prompt crafting.

- Iterate on the documentation request. If the AI's comments were too brief or missed important details, refine your prompt:

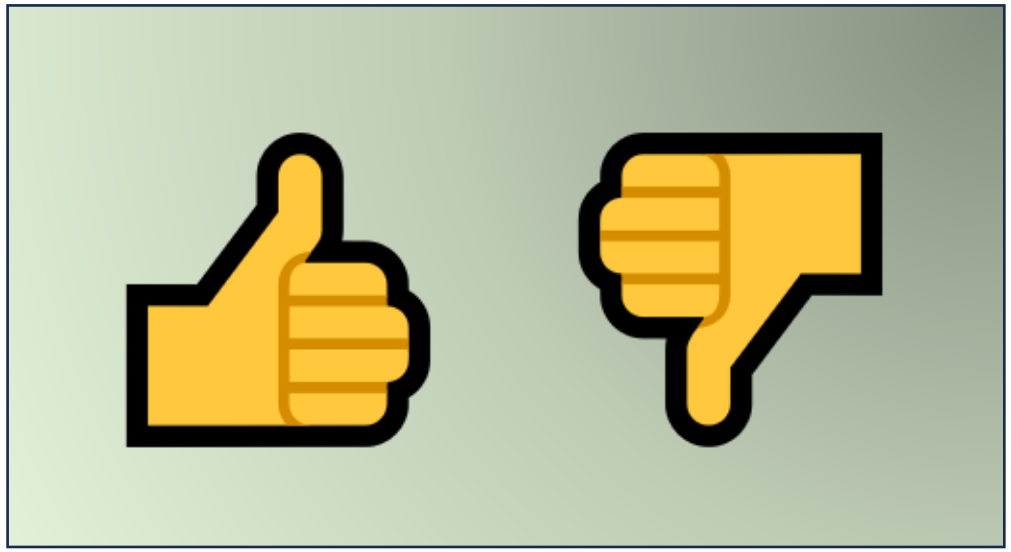
```
The comments you provided are a good start. Expand them to include a brief explanation in the class-level comment, parameter descriptions in all methods, and any preconditions or postconditions for each method.
```

- Refine your prompt for optimization. If the AI's initial response was vague or unsatisfactory, try a more specific prompt. Here’s a starting point, but try your own:

```
Suggest a specific optimization that would improve the time complexity of code. Please explain the optimization and show how to implement it in code.
```

Pulse Check

Were you able to get the AI to improve its output by asking it follow-up questions?



Q&A

If you have any questions, ask them now. Make sure you enter them in the Q&A widget, because that makes it easier for me to keep track of them.

5 minute break

Grab some water, check your email, have a look at your socials… we'll be back here in five minutes.

Part 2
Agentic Engineering in Practice

Delegating real work, and verifying what comes back

Welcome back

Think about the last time you looked at your own code and thought, “What was I doing here?!” Are there things you could have done to help future-you understand it faster?

Put your answers in the chat 

Introducing Claude Code

An AI agent that reads, writes, and runs your code

Get to know Claude Code

Claude Code is an AI agent that runs in your terminal and helps you write your code.

- It reads your files, edits them, runs commands, and reads the output, so it can carry out a whole task instead of just answering a question.
- It’s really good at generating boilerplate code, refactoring across multiple files, and explaining code you didn’t write, using Claude models trained on large datasets that include code.
- Beyond answering questions, you can prompt it for documentation, refactoring, or a whole new feature, and it makes the changes to your code for you. In manual mode it asks permission before each one.

Ways to work with Claude Code

- ⏩ **auto mode on** The default on Pro, Max, and Team plans. Claude Code edits files and runs commands without asking. A separate safety model checks each action.
- ⏸ **manual mode on** Press shift-tab once. Claude Code reads files on its own but asks your permission before it edits anything or runs most commands. Use this for today's exercises so you can see each change before it happens.
- ⏸ **plan mode on** Keep pressing shift-tab. Claude Code reads your code and answers, but can't change anything. Use it for questions, planning a feature, or debugging a logic error without changing code.
- ⏩ **bypass permissions on** Start with claude --dangerously-skip-permissions. Skips the normal permission prompts and safety checks. Anthropic recommends it only inside a container or VM.
- Type / for commands like /model and /init → type @ to point Claude Code at a specific file.

Giving Claude Code context

Claude Code doesn’t have to guess, because it can look where you point it or search your project until it finds what it needs.

Key ways to give it context:

- **Let it search:** The most important one. Claude Code will search your entire project (not just open files) to answer broad questions like "Where is the authentication logic?"
- **@Files:** Lets you pick specific files to "feed" to the AI (e.g., "Read @MathQuiz.cs and write a test for it").
- **CLAUDE.md:** A file in your project that Claude Code reads at the start of every session.

*Default to letting it search. Use **@** when you already know the file.*

Behind the scenes of Claude Code

- Claude Code supports multiple Claude models, and you can use /model to switch between them. For this course, the default model works well.
- Claude Code **searches your codebase on demand** rather than indexing it up front, so you can watch it look for exactly the files it needs to answer your question.
- You can interact with Claude Code through prompts, **/ commands** like /model and /init, or **@** to pull in specific files.
- In manual mode Claude Code asks your permission before it edits files or runs most commands. **Plan mode** makes it read-only, useful when you want analysis and answers without any changes.

Refactoring with Claude Code

Good things happen when you give Claude Code context. Use it today to make real improvements to your codebase.

Claude Code works best when your code communicates intent

- Claude Code generates code from your actual project. Every file it reads becomes part of its context.
- It pays attention to method and class names, filenames, comments, and the files it reads.
- The more your code communicates intent, the more useful Claude Code's work becomes.

Think of Claude Code as an **autonomous** AI coding partner. It's powerful, but it works best when your code gives it something meaningful to respond to. *It will always do what you said, not what you meant.*

How Claude Code figures out what you and your code are trying to do

Claude Code doesn't just guess. It looks at your code to build context. The more clues you give it, the better its work becomes.

Here are a few things Claude Code does to understand your code:

- Searches your project for the files it needs, then reads them.
- Uses file names, method names, variable names, and comments to infer purpose.
- It also sees the conversation so far, and every file it has already read.
- Picks up on patterns across your project, including naming conventions, function signatures, and code style.

Demo: Refactoring Code with Claude Code

- We can rely on **the *same specifying skills*** to help Claude Code generate better, more meaningful code.
- I'll start with a short piece of code that's extremely hard to read, with no clear names, no comments, and no context.
- Then I'll use **Claude Code** to refactor the real-world system this code is from and provide documentation.
- Claude Code needs context. Luckily, I found a document that will help it to suggest a much clearer, more readable refactoring.

Go to https://bit.ly/coding-ai-course
for the slides in this deck.

Exercise

Let's delegate a piece of work to Claude Code, and then get a second opinion on what it did.

Working with Claude Code means specifying and delegating

When you work with Claude Code, you describe what you want, the agent goes off and does it, and you check the result. That means the hard part of your work *is deciding what you want*, and figuring out *if the agent built it*.

- **You've done this before.** Delegation is what you're doing when you hand work to a teammate. You tell them what you need, give them enough context to do it well, and review what comes back.
- **Specify.** Say what you want and what "done" looks like. Vague requests get vague results, from people and from AI.

Delegate the work, then verify what comes back

- **Delegate.** Let it work. This is the part that feels strange at first, and it's where the leverage comes from.
- **Verify.** Read what it actually did, not just what it says it did. You're the one checking this into the codebase.
- Delegation is a skill, and we all do it. Some developers don't think of themselves as delegators, but you actually do it all the time. It just takes practice to get used to trusting the agent you're delegating to. But that's not always easy, because…
- …people can "read between the lines" and understand what you meant. Agents have a **do what I said, not what I meant** problem!

Tips for giving Claude Code the right context

What helps a new teammate understand your project helps Claude Code too:

- **Use meaningful names.** They tell Claude Code what you're building.
- **Write intentional comments.** Even quick notes steer what it does.
- **Keep your structure clear.** Organized code helps Claude Code follow your thinking.
- **Point it at the right files.** When you know which file matters, an @ reference saves it the search.
- **Write it down in CLAUDE.md.** The standing brief it reads at the start of every session.

Get a second opinion

The agent that wrote the code is the worst reviewer of it, because it's still carrying the context and assumptions that produced those choices.

- Open a second terminal and ask a fresh session to review the change. (/clear in the same window works too, but two windows let you compare.)
- **Show it what changed, not just the final code.** A fresh session has no history. Give it the original version so it can see exactly what changed.
- Ask what decisions the implementer made that the requirement didn't specify. "Is this good?" gets you compliments.
- A second opinion costs a terminal window and thirty seconds.

Exercise: Delegate a change

Let's delegate a change to Claude Code.

1. Go to https://bit.ly/coding-ai-course and copy the Python, Java, or C# version of the Greeter class we used in Part 1. Start Claude Code in an empty folder and press shift-tab until the status bar says manual mode on. Paste the class in and ask it to save it as a file.
2. Delegate the change with this prompt:

```
Update greet so it uses only the person's first name.
```

3. Now ask Claude Code to run greet with "Dr. Alice Chen", "Mary Ann Evans", and "Chen Wei". Is that what you meant?

Do you see any problems with the instruction we just gave to Claude Code?

Exercise: Review what it did

Now get a second opinion from an agent that wasn't in the room when the code was written.

1. Open a second terminal in the same folder and start a new Claude Code session. It has no memory of the conversation that made the change.
2. Tell it to read the Greeter file in this folder, paste in the original version from the course page, give it the requirement, then ask:

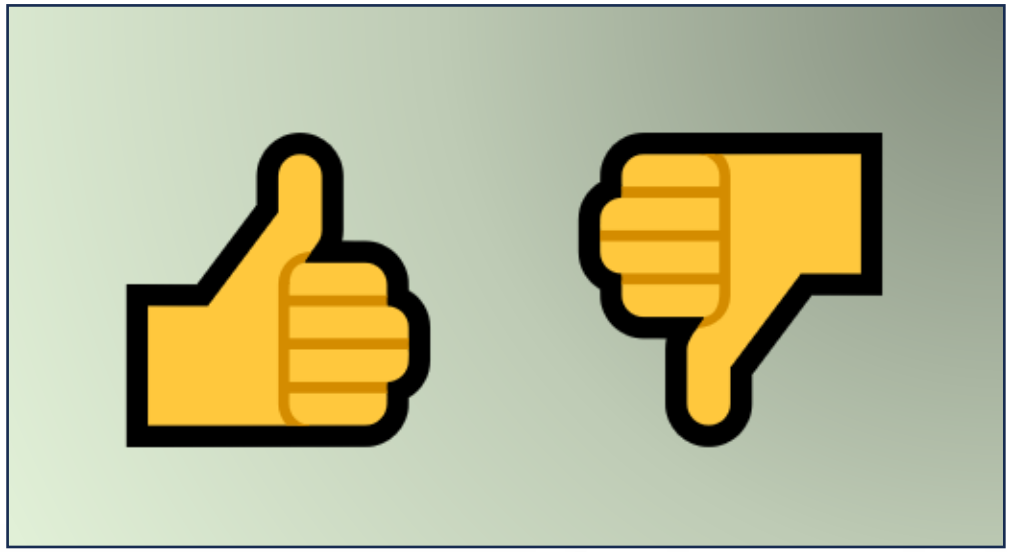
```
What decisions did the implementer have to make that
the requirement doesn't specify?
```

3. Compare what the reviewer found with what the first session told you it did.

Tip: set the reviewer's terminal to a different background color, with plenty of contrast, so you always know which window is which.

Pulse Check

Could you start using these techniques today to get better results from Claude Code on your own code?



Generating new code with Claude Code

You've specified, delegated, and verified. Now let's take it further and use Claude Code to generate full methods and whole projects.

Strategies for getting great results

Claude Code can make a really big difference in handling complex coding challenges, but it works best when you use it **strategically**:

- **Start with structure.** Describe the shape of what you want (the pieces, the names, the files) before you ask for the code.
- **Break ambiguous work into clear steps.** A clearly described step gets better results than a vague request of any size.
- **Be specific.** Use clear method names, well-named variables, and intentional comments to guide what it writes.
- **Try writing the *call* first.** Write the line that uses the method, then ask Claude Code to implement it.

Claude Code is a partnership. You're the one who decides.

Simple tips to get the most out of Claude Code and keep your code sharp.

- **You're the developer. Claude Code is just here to help.** Don't treat its output as correct until you've read, tested, and understood it.
- **Refine, don't just accept.** Treat what Claude Code wrote as a starting point, and then rewrite or improve as needed.
- **Give it context.** Good names, descriptive comments, and surrounding code all help Claude Code produce better results.
- **Iterate with it.** If the first result doesn't work, adjust your prompt or your code and try again. And again. Keep trying until you're happy with it.

Go to https://bit.ly/coding-ai-course
for the slides in this deck.

Exercise

Let's use Claude Code to create a simple project and explore its capabilities. To do this project you'll need Claude Code installed, along with the tools for your language.

⚠ Generating code is uncertain! ⚠

- **This exercise may take longer than the remaining course time. It's a take-home project for you to practice AI skills.**
- **Always review AI-generated code carefully.** Claude Code may sometimes produce code that doesn't compile or work as expected.
- **You're in charge!** Apply the best practices and critical thinking skills we've discussed throughout the course.
- **I'm still here to help.** If you encounter difficulties or want to see the process in action, a great way to get in touch with me is by raising an issue on the GitHub page for the course: https://bit.ly/coding-ai-course
- **Every AI experience is different.** If the first result isn't what you wanted, that's normal. Each run is different. Remember: iterate, iterate, iterate.

Get your environment ready for your language

- Claude Code runs in your terminal, so you don't need any IDE extensions. You do need Claude Code itself, plus the tools to build and run your language.
- Install Claude Code:
  - macOS or Linux: run curl -fsSL https://claude.ai/install.sh | bash
  - Windows PowerShell: run irm https://claude.ai/install.ps1 | iex
  - Run claude in any folder and sign in when it prompts you (you'll need a paid plan or an API account)
- Install the tools for your language:
  - Python: Python 3, and pytest for unit tests (pip install pytest)
  - Java: A JDK and Maven
  - C#: The .NET SDK

Create a new project

Start by creating a new project. Create a folder for your project and start Claude Code in it.

- **For Python:** Add a file called math_quiz.py.
  - Optionally, create a virtual environment with `python -m venv venv` and activate it.
  - For unit tests, you'll use pytest. Install it with: `pip install pytest`.
- **For Java:** Run mvn archetype:generate and choose maven-archetype-quickstart. Follow the prompts in the terminal.
- **For C#:** In your project folder, run the following:
  - `dotnet new console -n MathQuiz`
  - `dotnet new xunit -n MathQuiz.Tests`
  - `dotnet add MathQuiz.Tests reference MathQuiz`
  - *Note: you can change `xunit` to `nunit` or `mstest` if you prefer a different unit test framework*

Prompt Claude Code to build the app

Type this prompt into Claude Code. It will create or update your code files directly:

```
Create a console application that quizzes the user with random
math problems. Generate two random numbers from 1 to 9 and pick
either addition or multiplication. Prompt for an answer,
congratulate if correct, retry if incorrect. Exit if user enters
a non-numeric value. Put the game in a class that is testable,
with public methods to generate operators, numbers, and
questions.
```

You can also try the same prompt with a different model using /model. Does it match?

Carefully review Claude Code's output

- Review each change carefully before you approve it. Make sure the class has public methods that can be tested, including a method that generates a question.
- Approve the changes, then run the app and make sure it works.
- If it's not what you wanted, say no at the permission prompt (or ask it to undo the change) and revise the prompt. Iterate until you have a solution you like.
- **Get a second opinion.** A fresh session that reads the code and the prompt will catch things the session that wrote it won't.

Generate unit tests

Ask Claude Code to create unit tests with this prompt:

```
Create unit tests for the math quiz game. Test random number generation, operation selection, and answer checking.
```

Read the tests before you run them. Here are a few it may have skipped:

```
Add a test that generates 100 questions and checks that every number is between 1 and 9.
```

```
Add a test that checks the question text matches the numbers and operator that were generated.
```

```
Add a test that checks a correct answer is accepted and a wrong answer is rejected.
```

Vibe code your tests

Vibe coding, which means rapidly prompting, getting code, and iterating, is a fast way to get tests building and running. Use it for the first draft, then read what each test checks before you trust a green run.

- Make sure the code builds with no problems.
- If there are errors that keep it from building, ask Claude Code to fix them. It will read the errors and keep working until your code builds.
- Then run your tests (we'll show you how on the next slide). If there are any failures, ask Claude Code to fix those too.

Run the unit tests in a terminal window

- Open a second terminal and run the tests (or ask Claude Code to run them for you):
  - Java: Run mvn test
  - C#: Run dotnet test in the MathQuiz.Tests folder
  - Python: Run pytest
- If no tests are displayed, or if there's an error, make sure your code builds without problems.
- If a test doesn't pass, ask the AI to help you fix it. Give it this prompt:

  Why didn't this test pass?
- Decide whether the test or the code is wrong, and fix that one.

Ask the AI to test edge cases

Let's add more tests using prompts.

Ask Claude Code to add a test for an edge case. Use this prompt:

```
Add a unit test to check an edge case
```

Review the test it wrote, run it, and make sure it passes.

Now ask for more. Use this prompt:

```
Add unit tests for additional edge cases
```

Review the tests it wrote, run them, and make sure they pass. Fix any errors.

Document and explain the code

- Ask Claude Code to add documentation to your main code file. Use one of these prompts:

Java: `Add JavaDoc comments to all methods`

C#: `Add XMLDoc comments to all methods`

Python: `Add docstrings to all functions and classes`

- Review the generated documentation and make any necessary adjustments.
- Ask Claude Code to explain all the code in your app. Try a few different ways of phrasing that prompt, see what works best.

Use AI to modify your code

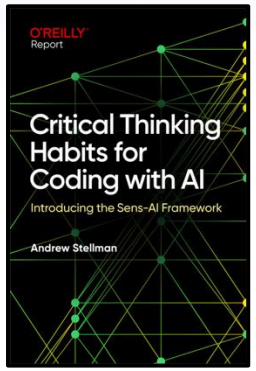
Ask Claude Code to make changes to your code. Here are a few things to try:

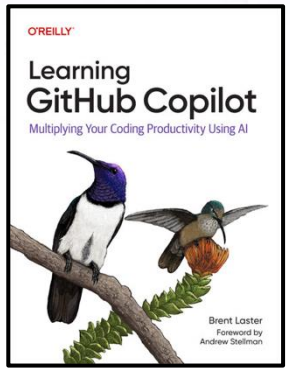
- Split the code that generates the question into two methods or functions, one to generate a question and one to check the answer. Add unit tests for both.
- Modify the app so it keeps track of how many questions in a row the player gets right. Remember the longest question streak. Implement this with public methods and generate unit tests for those methods.
- Implement robust error handling and input validation. Create a separate method to validate user input, ensuring it handles various edge cases (e.g., non-numeric input, extremely large numbers, negative numbers). Generate unit tests for this validation method, including tests for different types of invalid input.

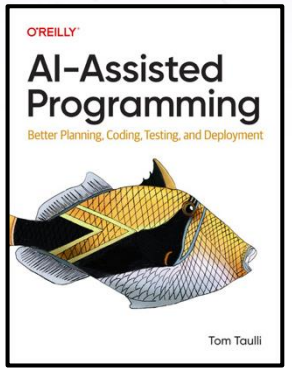
Q&A

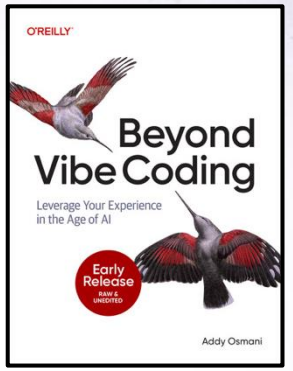
If you have any questions, enter them in the Q&A widget.

Additional resources









Want to dive deeper into AI-assisted coding and prompt engineering? Here are some great places to start.

- "Critical Thinking Habits for Coding with AI" by Andrew Stellman (O'Reilly, 2024) https://learning.oreilly.com/library/view/critical-thinking-habits/0642572243326/
- "Learning GitHub Copilot" by Brent Laster (O'Reilly, 2025) – this book is an amazing resource for learning to use Copilot, and **I wrote the foreword for it**: https://learning.oreilly.com/library/view/learning-github-copilot/9781098164645/
- "AI-Assisted Programming" by Tom Taulli (O'Reilly, 2024) – another fantastic resource, and the chapter on prompt engineering is a phenomenal way to follow up on this course: https://learning.oreilly.com/library/view/ai-assisted-programming/9781098164553/
- "Beyond Vibe Coding" by Addy Osmani (O'Reilly, 2025) https://learning.oreilly.com/library/view/beyond-vibe-coding/9798341634749/

linkedin.com/in/andrewstellman/ github.com/andrewstellman Andrew Stellman.bsky.social

Thank you!

I hope you enjoyed my course! If you want to learn more, follow me on social media, and check out *Head First C#* and my other books on O'Reilly Learning.

Thanks so much for attending!